

## **LAPORAN**

### **Sistem Informasi Perpustakaan Digital**

Laporan ini disusun untuk memenuhi salah satu penilaian Ujian Tengah Semester mata kuliah peminatan Pembangunan Perangkat Lunak Berorientasi Service

Dosen Pengampu:

Muhammad Panji Muslim, S.Pd., M.Kom



Disusun Oleh:

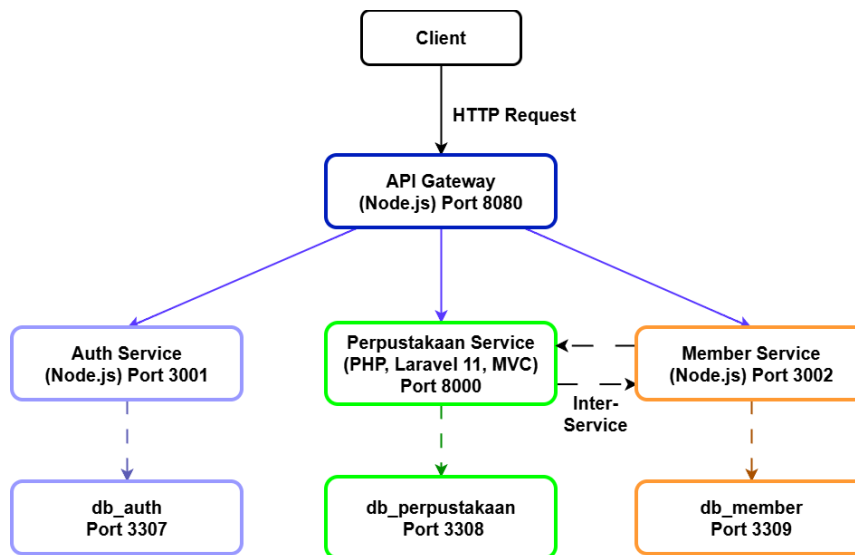
Akbar Fitri Andhika 2410511011

**PROGRAM STUDI S1 INFORMATIKA FAKULTAS ILMU KOMPUTER  
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAKARTA**

**2026**

## Arsitektur Sistem

Sistem terdiri dari empat komponen yang berjalan dalam jaringan Docker internal, dengan diagram arsitektur dari program ini bisa dilihat pada gambar berikut:



Pada program yang dibuat tidak menggunakan sistem monolitik tetapi berbasis service atau dengan melakukan pemisahan setiap servicenya. Pada program ini memiliki tiga service yaitu terdapat auth-service, perpustakaan-service, dan member-service.

1. auth-service dipisahkan karena autentikasi merupakan domain dengan tingkat sensitivitas yang tinggi. Service ini perlu diaudit, dipantau, dan diperbarui secara independen tanpa memengaruhi fitur lainnya. Satu JWT secret dikonfigurasi di semua service tanpa harus mengekspos kode autentikasi.
2. perpustakaan-service dipisahkan karena katalog buku adalah fitur yang bersifat publik dan read-heavy yaitu dapat diakses tanpa login oleh siapapun. Pola akses ini sangat berbeda dibandingkan dengan transaksi peminjaman yang write-heavy dan membutuhkan konsistensi yang tinggi. Pemisahan dilakukan karena service ini diskalakan secara horizontal untuk melayani banyak pembaca katalog tanpa memengaruhi service transaksi.
3. member-service dipisahkan karena domain transaksi yaitu pinjaman dan denda, yang memiliki logika bisnis yang kompleks. Dengan memperhitungkan denda harian, pembatasan jumlah pinjaman aktif, validasi status anggota. Memisahkan service ini memastikan perubahan aturan bisnis peminjaman tidak memengaruhi kode manajemen buku.

## SERVICE

### 1. Auth Service

Auth service dibuat dengan menggunakan Node.js, Express, Sequelize, dan MySQL bertanggung jawab atas seluruh urusan mengenai identitas yang menggunakan program, seperti registrasi, login, pembuatan token, refresh token, logout, dan login alternatif melalui GitHub OAuth. Dengan database db\_auth yang memiliki 2 tabel yaitu:

Tabel users

| Kolom         | Tipe                     | Keterangan                 |
|---------------|--------------------------|----------------------------|
| id            | UUID (PK)                | Primary key auto-generated |
| nama          | VARCHAR (100)            | Nama lengkap pengguna      |
| email         | VARCHAR (150) UNIQUE     | Email unik                 |
| hash_password | VARCHAR (100) NULL       | NULL jika daftar via OAuth |
| provier_oauth | ENUM('lokal', 'github')  | Asal pendaftaran           |
| id_oauth      | VARCHAR (100) NULL       | ID dari provider OAuth     |
| url_foto      | TEXT NULL                | Foto profil dari GitHub    |
| peran         | ENUM('admin', 'anggota') | Hak akses                  |
| dibuat_pada   | TIMESTAMP                | Waktu registrasi           |

Tabel token\_refresh

| Kolom           | Tipe              | Keterangan                      |
|-----------------|-------------------|---------------------------------|
| id              | UUID (PK)         | Primary key                     |
| id_pengguna     | UUID (FK → users) | Pemilik token                   |
| token           | TEXT              | Nilai refresh token             |
| kadaluarsa_pada | DATETIME          | Waktu kedaluwarsa               |
| dicabut         | BOOLEAN           | True = sudah logout (blacklist) |
| dibuat_pada     | TIMESTAMP         | Waktu pembuatan                 |

Endpoint Auth Service

| Method | Path                            | Auth | Keterangan                      |
|--------|---------------------------------|------|---------------------------------|
| POST   | /api/auth/register              | -    | Daftar akun baru                |
| POST   | /api/auth/login                 | -    | Login email + password          |
| POST   | /api/auth/refresh               | -    | Perbarui access token           |
| POST   | /api/auth/logout                | -    | Cabut refresh token (blacklist) |
| GET    | /api/auth/me                    | JWT  | Data pengguna aktif             |
| GET    | /api/auth/oauth/github          | -    | Redirect ke GitHub OAuth        |
| GET    | /api/auth/oauth/github/callback | -    | Callback dari GitHub            |

## 2. Perpustakaan Service

Perpustakaan service dibuat dengan menggunakan PHP Laravel 11 (PHP MVC) dan MySQL, service ini bertanggung jawab untuk mengelola katalog buku, kategori, dan ulasan. Dibangun dengan Laravel 11 mengikuti pola MVC yang ketat yaitu model harus berada di app/Models/, Controller di app/Http/Controllers/, dan routing di routes/api.php. Database db\_perpustakaan dengan memiliki 6 tabel.

Tabel kategori

| Kolom         | Tipe                | Keterangan                 |
|---------------|---------------------|----------------------------|
| id            | BIGINT (PK)         | Primary key auto increment |
| nama_kategori | VARCHAR(100) UNIQUE | Nama kategori              |
| slug          | VARCHAR(120) UNIQUE | Slug URL                   |
| untuk_usia    | INTEGER NULL        | Rekomendasi usia baca      |

Tabel buku

| Kolom           | Tipe                    | Keterangan                 |
|-----------------|-------------------------|----------------------------|
| id              | BIGINT (PK)             | Primary key auto increment |
| kategori_id     | FK → kategori           | Kategori buku              |
| judul           | VARCHAR(255)            | Judul buku                 |
| isbn            | VARCHAR(20) UNIQUE NULL | ISBN                       |
| jumlah_halaman  | INTEGER NULL            | Jumlah halaman             |
| deskripsi       | TEXT NULL               | Deskripsi buku             |
| tahun_terbit    | INTEGER NULL            | Tahun terbit               |
| penerbit        | VARCHAR(150) NULL       | Nama penerbit              |
| url_sampul      | VARCHAR(500) NULL       | URL gambar sampul          |
| jumlah_total    | INTEGER                 | Total eksemplar            |
| jumlah_tersedia | INTEGER                 | Stok tersedia              |

Tabel author\_buku

| Kolom       | Tipe         | Keterangan                 |
|-------------|--------------|----------------------------|
| id          | BIGINT (PK)  | Primary key auto increment |
| buku_id     | FK → buku    | Relasi ke buku             |
| nama_author | VARCHAR(150) | Nama penulis               |

Tabel stok\_buku

| Kolom      | Tipe        | Keterangan                      |
|------------|-------------|---------------------------------|
| id         | BIGINT (PK) | Primary key auto increment      |
| buku_id    | FK → buku   | Relasi ke buku (cascade delete) |
| created_at | TIMESTAMP   | Waktu dibuat                    |
| updated_at | TIMESTAMP   | Waktu diperbarui                |

Tabel tag\_buku

| Kolom      | Tipe        | Keterangan                      |
|------------|-------------|---------------------------------|
| id         | BIGINT (PK) | Primary key auto increment      |
| buku_id    | FK → buku   | Relasi ke buku (cascade delete) |
| nama_tag   | VARCHAR(80) | Nama tag buku                   |
| created_at | TIMESTAMP   | Waktu dibuat                    |
| updated_at | TIMESTAMP   | Waktu diperbarui                |

Tabel ulasan\_buku

| Kolom       | Tipe        | Keterangan                      |
|-------------|-------------|---------------------------------|
| id          | BIGINT (PK) | Primary key auto increment      |
| buku_id     | FK → buku   | Relasi ke buku (cascade delete) |
| id_pengguna | UUID        | ID pengguna dari auth-service   |

|               |              |                        |
|---------------|--------------|------------------------|
| nama_pengguna | VARCHAR(100) | Snapshot nama pengguna |
| rating        | TINYINT      | Nilai 1–5              |
| komentar      | TEXT NULL    | Isi ulasan             |
| created_at    | TIMESTAMP    | Waktu dibuat           |
| updated_at    | TIMESTAMP    | Waktu diperbarui       |

### Endpoint Perpustakaan Service

| Method | Path                             | Auth  | Keterangan                             |
|--------|----------------------------------|-------|----------------------------------------|
| GET    | /api/buku                        | -     | Katalog publik (paging + filter)       |
| GET    | /api/buku/{id}                   | -     | Detail buku                            |
| POST   | /api/buku                        | Admin | Tambah buku baru                       |
| PUT    | /api/buku/{id}                   | Admin | Update buku                            |
| DELETE | /api/buku/{id}                   | Admin | Hapus buku                             |
| PATCH  | /api/buku/{id}/ketersediaan      | JWT   | Update stok (dipanggil member-service) |
| GET    | /api/kategori                    | -     | List kategori                          |
| POST   | /api/kategori                    | Admin | Tambah kategori                        |
| DELETE | /api/kategori/{id}               | Admin | Hapus kategori                         |
| GET    | /api/buku/{id}/ulasan            | -     | Ulasan publik                          |
| POST   | /api/buku/{id}/ulasan            | JWT   | Tambah ulasan                          |
| DELETE | /api/buku/{id}/ulasan/{ulasanId} | JWT   | Hapus ulasan                           |

Pada endpoint GET /api/buku terdapat paging dan filtering yang dipanggil melalui parameter halaman, per\_halaman, dan filtering berdasarkan judul (cari), kategori (kategori\_id), dan tahun terbit (tahun\_terbit). Seluruh parameter bersifat opsional dan dapat dikombinasikan sesuai kebutuhan.

Pada service ini juga menerapkan Inter-Service yaitu antara perpustakaan-service dengan member-service. Jadi saat anggota ingin memproses peminjaman buku, perpustakaan-service terlebih dahulu akan melakukan verifikasi pinjaman aktif anggota melalui kelas MemberServiceClient dengan memanggil endpoint GET /api/anggota/{id}/pinjaman/aktif pada member-service. Pendekatan ini diperlukan karena data pinjaman berada di domain member-service dengan database yang terpisah, sehingga tidak dapat diakses secara langsung. Pemanggilan dilakukan dengan meneruskan token JWT pada header Authorization agar identitas pemanggil dapat diverifikasi. Apabila terdapat pinjaman aktif yang belum diselesaikan, perpustakaan-service akan menolak permintaan peminjaman baru.

### 3. Member Service

Member service dibuat dengan Node.js, Express, Sequelize, dan MySQL. Service ini bertanggung jawab untuk mengelola data anggota perpustakaan, transaksi peminjaman, dan denda keterlambatan. Dengan database db\_member yang memiliki 3 tabel

Tabel anggota

| Kolom       | Tipe                | Keterangan                    |
|-------------|---------------------|-------------------------------|
| id          | UUID (PK)           | Primary key                   |
| id_pengguna | UUID UNIQUE         | ID pengguna dari auth-service |
| nama        | VARCHAR(100)        | Nama anggota                  |
| email       | VARCHAR(150) UNIQUE | Email                         |
| telepon     | VARCHAR(20) NULL    | No. telepon                   |
| alamat      | TEXT NULL           | Alamat                        |
| aktif       | BOOLEAN             | Status keanggotaan            |

Tabel pinjaman

| Kolom               | Tipe                                     | Keterangan                                          |
|---------------------|------------------------------------------|-----------------------------------------------------|
| id                  | UUID (PK)                                | Primary key                                         |
| id_anggota          | FK → anggota                             | Pemilik pinjaman                                    |
| id_buku             | INTEGER                                  | ID buku dari perpustakaan-service                   |
| judul_buku          | VARCHAR(255)                             | Snapshot judul (agar tidak bergantung service lain) |
| tanggal_pinjam      | DATE                                     | Tanggal mulai pinjam                                |
| tanggal_jatuh_tempo | DATE                                     | Batas pengembalian                                  |
| tanggal_kembali     | DATE NULL                                | Tanggal dikembalikan                                |
| status              | ENUM('aktif','dikembalikan','terlambat') | Status pinjaman                                     |

Tabel denda

| Kolom         | Tipe                 | Keterangan               |
|---------------|----------------------|--------------------------|
| id            | UUID (PK)            | Primary key              |
| id_pinjaman   | FK → pinjaman UNIQUE | Satu pinjaman satu denda |
| jumlah        | INTEGER              | Nominal denda (Rp)       |
| sudah_dibayar | BOOLEAN              | Status pembayaran        |
| dibayar_pada  | DATETIME NULL        | Waktu pelunasan          |

Endpoint Member Service

| Method | Path                             | Auth  | Keterangan                           |
|--------|----------------------------------|-------|--------------------------------------|
| GET    | /api/anggota                     | Admin | List semua anggota (paging + search) |
| GET    | /api/anggota/{id}                | JWT   | Detail anggota                       |
| POST   | /api/anggota                     | JWT   | Daftar sebagai anggota               |
| PUT    | /api/anggota/{id}                | JWT   | Update profil anggota                |
| GET    | /api/anggota/{id}/pinjaman/aktif | JWT   | Inter-service: pinjaman aktif        |
| GET    | /api/pinjaman                    | JWT   | List pinjaman                        |
| POST   | /api/pinjaman                    | JWT   | Buat pinjaman baru                   |
| PATCH  | /api/pinjaman/{id}/kembali       | JWT   | Proses pengembalian                  |
| GET    | /api/denda                       | JWT   | List denda                           |

|       |                       |     |             |
|-------|-----------------------|-----|-------------|
| PATCH | /api/denda/{id}/bayar | JWT | Bayar denda |
|-------|-----------------------|-----|-------------|

API Gateway dibuat dengan Node.js dan Express. Gateway adalah single entry point seluruh sistem. Klien (Postman maupun frontend) hanya boleh memanggil melalui gateway di port 8080, tidak boleh langsung ke service belakang. Adapun tiga fungsi gateway pada program ini yaitu

1. Routing per service yang dilakukan pada setiap path prefix yang diarahkan ke service yang sesuai dengan menggunakan http-proxy-middleware.

| Path Prefix                                           | Service Tujuan                     | Port |
|-------------------------------------------------------|------------------------------------|------|
| /api/auth/register, /login, /refresh, /logout, /oauth | auth-service                       | 3001 |
| /api/auth/* (lainnya)                                 | auth-service (terproteksi)         | 3001 |
| GET /api/buku, GET /api/buku/:id, GET /api/kategori   | perpustakaan-service (publik)      | 8000 |
| /api/buku/, /api/kategori/ (lainnya)                  | perpustakaan-service (terproteksi) | 8000 |
| /api/anggota/, /api/pinjaman/, /api/denda/*           | member-service                     | 3002 |

2. JWT Validation di sisi gateway, jadi sebelum request diteruskan ke service, gateway akan memverifikasi JWT dengan menggunakan jsonwebtoken. Jika token tidak valid atau sudah expired, maka gateway akan langsung menolak dengan response 401 tanpa meneruskan ke service belakang.
3. Rate Limiting dengan pengimplementasi in-memory per IP. Jadi setiap IP akan dibatasi dengan maksimal 60 request per menit. Jika melampaui batas tersebut, gateway akan merespons dengan status 429.

## Otentikasi JWT

Pada program ini sistem menggunakan dua jenis mekanisme token

- Access Token dengan JWT yang ditandatangani dengan JWT\_SECRET, yang berlaku selama 15 menit. Digunakan untuk mengakses endpoint terproteksi.
- Refresh Token dengan JWT yang ditandatangani dengan JWT\_REFRESH\_SECRET, yang berlaku selama 7 hari. Digunakan untuk mendapatkan access token baru tanpa login ulang. Disimpan di database tabel token\_refresh.

Jadi token mengalami rotasi yaitu ketika endpoint /api/auth/refresh dipanggil, sistem tidak hanya menerbitkan access token baru, tetapi refresh token lama juga dicabut (dicabut = true) dan refresh token baru diterbitkan. Ini mencegah penyalahgunaan token yang bocor.

Kemudian saat Logout endpoint /api/auth/logout menerima refresh\_token lalu menandai token tersebut sebagai dicabut = true di database. Selanjutnya token yang sudah dicabut tidak dapat digunakan untuk refresh meskipun belum melewati waktu kedaluwarsa 7 hari.

Lalu middleware JWT akan melakukan verifikasi JWT yang diimplementasikan sebagai middleware terpusat, tidak manual per-endpoint, yaitu dengan `middleware/jwtMiddleware.js` dengan fungsi `verifikasiToken` pada `auth-service` dan `member-service`. Pada `perpustakaan-service` `app/Http/Middleware/JwtVerify.php` di daftarkan di `routes/api.php` sebagai `middleware('jwt.verify')`. Dan pada API Gateway `verifikasiToken` diaplikasikan sebelum proxy, sehingga request invalid tidak pernah sampai ke service belakang.

### **Otorisasi OAuth 2.0 dengan GitHub**

Pada program ini sistem mengimplementasikan login alternatif melalui GitHub dengan menggunakan Authorization Code Flow, yaitu alur OAuth 2.0 yang paling aman karena access tokennya tidak pernah terekspos langsung ke browser secara langsung. Jadi ketika pengguna mengakses `GET /api/auth/oauth/github`, request akan diterima oleh Gateway lalu diteruskan ke `auth-service`. Kemudian pada `auth-service` akan mengarahkan pengguna ke halaman login resmi GitHub di `https://github.com/login/oauth/authorize` beserta parameter `client_id`, `redirect_uri`, dan `scope=user:email`. Di sinilah pengguna akan memasukkan kredensial GitHub mereka secara langsung dengan aplikasi tidak pernah melihat username maupun password GitHub pengguna.

Setelah pengguna berhasil login dan menyetujui izin akses yang diberikan, GitHub akan mengarahkan kembali ke callback URL yang telah didaftarkan dengan menyertakan parameter `code` yang bersifat sementara dan hanya dapat digunakan satu kali. Kode ini kemudian akan ditukar oleh `auth-service` secara server-to-server ke endpoint `https://github.com/login/oauth/access_token` untuk mendapatkan access token GitHub. Proses pertukaran ini terjadi di sisi server, bukan di browser, sehingga access token GitHub tidak pernah terekspos ke klien.

Dengan access token GitHub yang diperoleh, `auth-service` akan mengambil data profil pengguna dari `https://api.github.com/user` sekaligus daftar email dari `https://api.github.com/user/emails` untuk mendapatkan email utama yang sudah diverifikasi. Setelah data pengguna berhasil diperoleh, lalu `auth-service` akan mencari atau membuat akun lokal di database, dengan menerbitkan JWT sistem sendiri, dan mengarahkan pengguna kembali ke frontend dengan `access_token` dan `refresh_token` sebagai query parameter.

Pada sistem ini juga melakukan pemetaan ke user local, jadi ketika callback dari GitHub diterima, sistem menjalankan tiga skenario pemetaan secara berurutan untuk memastikan tidak terjadi duplikasi akun.

1. Pada scenario pertama, jika pengguna sudah pernah login melalui GitHub sebelumnya, sistem akan mencari akun berdasarkan kombinasi `id_oauth` atau ID unik GitHub dan `provider_oauth = 'github'`. Karena ID GitHub bersifat tetap dan unik, jadi pencarian ini akan menjamin akun yang ditemukan adalah akun yang benar. Apabila ditemukan, pengguna secara langsung akan diarahkan ke proses penerbitan JWT tanpa langkah tambahan.
2. Pada scenario kedua, jika tidak ditemukan akun dengan `id_oauth` tersebut, sistem akan memeriksa apakah email utama dari GitHub sudah terdaftar sebagai akun

local, yaitu akun yang didaftarkan dengan email dan password. Apabila ditemukan, sistem akan menghubungkan akun lokal yang ada dengan akun GitHub dengan memperbarui kolom provider\_oauth, id\_oauth, dan url\_foto. Skenario ini akan menangani kasus di mana pengguna sebelumnya sudah mendaftar secara manual dan kini ingin login menggunakan GitHub dengan email yang sama.

3. Pada scenario ketiga, jika tidak ditemukan akun sama sekali, baik berdasarkan id\_oauth ataupun email, sistem akan membuat akun baru secara otomatis dengan data yang diperoleh dari GitHub. Akun baru ini akan secara langsung ditandai dengan provider\_oauth = 'github' tanpa password karena pengguna tidak mendaftarkan password secara manual.

Data yang disimpan dari GitHub mencakup nama yang diambil dari field name pada profil GitHub, atau menggunakan login dengan username sebagai fallback, apabila field name kosong, yang akan diambil adalah email dari daftar email pengguna GitHub dengan memilih yang berstatus primary dan verified, serta url\_foto yang diambil dari field avatar\_url sebagai foto profil dari pengguna.

### **Komunikasi Antar Service (Inter-Service)**

1. member-service ke perpustakaan-service

Pada komunikasi ini terjadi dalam dua situasi yaitu saat melakukan peminjaman dan pengembalian buku. Ketika anggota ingin meminjam buku, member-service memanggil endpoint PATCH /api/buku/{id}/ketersediaan dengan body {"aksi": "pinjam"} ke perpustakaan-service. Perpustakaan-service kemudian akan memeriksa ketersediaan stok, apabila habis maka akan dikembalikan status dengan respons 409 dan peminjaman dibatalkan. Apabila tersedia nilai jumlah\_tersedia akan dikurangi satu. Proses sebaliknya terjadi saat pengembalian buku dengan body {"aksi": "kembali"} sehingga nilai jumlah\_tersedia akan bertambah kembali. Dengan mekanisme ini, data stok buku selalu mencerminkan kondisi aktual meskipun transaksi dikelola oleh service yang berbeda.

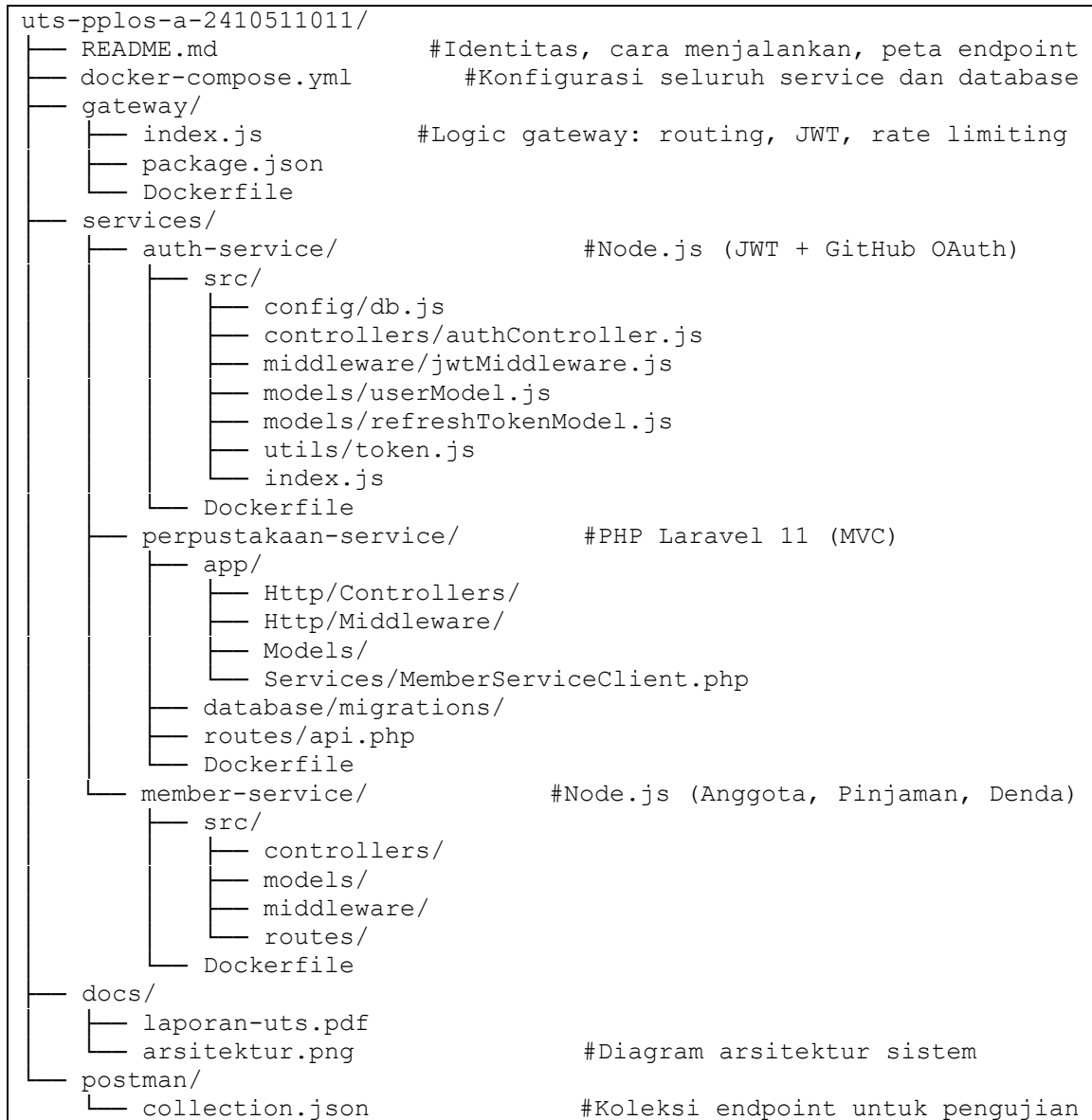
2. perpustakaan-service ke member-service

Lalu pada kasus ini terjadi saat sebelum menyetujui untuk peminjaman buku baru, perpustakaan-service terlebih dahulu akan memvalidasi apakah anggota masih memiliki pinjaman aktif yang belum dikembalikan. Karena data pinjaman berada di database member-service yang terpisah, validasi akan dilakukan melalui pemanggilan GET /api/anggota/{id}/pinjaman/aktif dengan menggunakan kelas MemberServiceClient. Apabila terdapat pinjaman aktif, maka permintaan peminjaman baru ditolak hingga seluruh pinjaman diselesaikan terlebih dahulu.

Pada kedua kasus diatas komunikasi dilakukan dengan meneruskan token JWT pengguna pada header Authorization: Bearer {token}. Sehingga service tujuan dapat langsung melakukan verifikasi identitas pemanggil dengan menggunakan JWT secret yang dibagikan antar service, tanpa perlu memanggil auth-service secara terpisah pada setiap request inter-service.

## Struktur Repository

Struktur direktori repository disusun sebagai berikut:



Setiap service memiliki Dockerfile yang digunakan untuk keperluan containerisasi, dan seluruh service dikorelasikan dengan menggunakan docker-compose.yml di root repository. Folder docs menyimpan laporan dan diagram arsitektur dari proyek yang dikerjakan, dan folder postman digunakan untuk menyimpan koleksi endpoint yang dapat diimpor langsung ke aplikasi Postman untuk keperluan pengujian.

## HTTP Method dan Response Code

Pada sistem ini menerapkan HTTP method secara semantik, yaitu

| Method | Penggunaan                          |
|--------|-------------------------------------|
| GET    | Mengambil data (list maupun detail) |

|        |                                                                            |
|--------|----------------------------------------------------------------------------|
| POST   | Membuat resource baru                                                      |
| PUT    | Memperbarui seluruh data resource                                          |
| PATCH  | Memperbarui sebagian data (ketersediaan buku, status kembali, bayar denda) |
| DELETE | Menghapus resource                                                         |

Dengan Response code yang digunakan:

| Kode | Situasi                                             |
|------|-----------------------------------------------------|
| 200  | Request berhasil (GET, POST login/refresh/logout)   |
| 201  | Resource berhasil dibuat                            |
| 400  | Input tidak lengkap atau tidak valid                |
| 401  | Token tidak ada, tidak valid, atau expired          |
| 403  | Token valid tapi tidak punya hak akses              |
| 404  | Resource tidak ditemukan                            |
| 409  | Konflik data (email sudah terdaftar, stok habis)    |
| 422  | Validasi gagal (format salah, panjang tidak sesuai) |
| 429  | Rate limit terlampaui                               |
| 500  | Kesalahan internal server                           |
| 502  | Service backend tidak tersedia (dari gateway)       |

## Kesimpulan

Proyek Sistem Informasi Perpustakaan Digital berhasil dibangun menggunakan arsitektur microservice dengan empat komponen utama, yaitu auth-service (Node.js), perpustakaan-service (Laravel 11 PHP MVC), member-service (Node.js), dan API Gateway (Node.js). Dengan setiap servicenya memiliki database MySQL masing-masing dan berkomunikasi melalui REST API.

Sistem berhasil mengimplementasikan:

1. Autentikasi JWT dengan access token 15 menit dan refresh token 7 hari, yang dilengkapi dengan mekanisme token blacklist saat logout dan rotasi token saat refresh.
2. Login alternatif melalui GitHub OAuth 2.0 dengan menggunakan Authorization Code Flow dengan pemetaan ke user lokal di database.
3. API Gateway sebagai single entry point yang melakukan routing, melakukan validasi JWT terpusat, dan rate limiting 60 request/menit per IP.
4. Komunikasi inter-service antara perpustakaan-service dan member-service untuk sinkronisasi stok buku dan validasi pinjaman aktif.
5. Paging dan filtering pada endpoint listing di perpustakaan-service dan member-service.

Dibandingkan menggunakan pendekatan monolitik, arsitektur microservice yang diterapkan memberikan keuntungan berupa kemampuan scaling independen per servicenya, mengisolasi kegagalan antar domain, dan fleksibilitas pemilihan teknologi yang berbeda, seperti Node.js dan PHP dalam satu sistem yang terintegrasi.